

Работа с файловой системой

Теория:

1) Фленов М.Е. Библия C#. - Санкт-Петербург : БХВ-Петербург, 2019.

<https://ibooks.ru/reading.php?productid=366634> п.14.2-14.4

2) <https://metanit.com/sharp/tutorial/> - Глава 22. Работа с файловой системой

Для работы с файлами и файловыми потоками применяется пространство имен **System.IO**:
using System.IO;

1 Работа с дисками

Класс DriveInfo – предоставляет доступ к сведениям о диске.

DriveInfo.GetDrives(); – статический метод, возвращающий набор всех логических дисков компьютера.

Свойства объектов класса (доступ к информации о конкретном диске):

- **Name:** имя диска (например, C:\ D:\)
- **VolumeLabel:** метка тома (например, System)
- **IsReady:** готов ли диск (например, DVD может быть не вставлен в дисковод)
- **DriveFormat:** имя файловой системы (например: FAT, NTFS)
- **DriveType:** тип диска (например: Fixed (жесткий диск), Removable (флэшка), CDRom (CD или DVD))
- **TotalSize:** общий размер диска в байтах
- **TotalFreeSpace:** общий объем свободного места на диске в байтах
- **AvailableFreeSpace:** объем доступного пользователю свободного места на диске в байтах

Пример 1. Вывод информации о всех устройствах (жестких дисках, флешках и т.д.)

Добавить на форму метку с названием drivesLabel. На загрузку формы прописать следующий код:

```
drivesLabel.Text = "";
DriveInfo[] drives = DriveInfo.GetDrives();
foreach (var item in drives)
{
    drivesLabel.Text += $"Устройство: {item.Name} " + // имя устройства
        $"тип: {item.DriveFormat} " + // файловая система
        $"размер: {item.TotalSize/1024/1024} МБ" + // размер, переведенный в МБ
        $"{Environment.NewLine}"; // перевод на новую строку
}
```

2 Работа с каталогами (папками)

Класс Directory – предоставляет набор статических методов для управления каталогом и получения информации о каталоге.

Общая форма вызова методов:

Directory.ИмяМетода("имя папки", параметры метода)

Первым параметром передается путь к папке (имя папки), например: "C:\\Temp" или @"C:\\Temp".

Класс DirectoryInfo – предоставляет набор методов экземпляра для управления каталогом и свойств для получения информации о каталоге.

Для использования обязательно создать объект класса:

DirectoryInfo имяОбъекта = new DirectoryInfo("имя папки");

имяОбъекта.ИмяМетода(параметры метода) или **имяОбъекта.ИмяСвойства**

Имя папки указано при создании объекта, поэтому в параметрах метода не указывается

Примеры методов и свойств классов Directory и DirectoryInfo и их вызовов

Действие	Применение Directory	Применение DirectoryInfo
Предварительное действие для работы примеров	Создать строковую переменную path для хранения пути к папке: <code>string path = @"C:\Temp";</code>	Создать переменную di типа DirectoryInfo: <code>string path = @"C:\Temp"; DirectoryInfo di = new DirectoryInfo(path);</code>
Создание папки	<code>Directory.CreateDirectory(path);</code>	<code>di.Create();</code>
Удаление папки	<code>Directory.Delete(path);</code>	<code>di.Delete();</code>
Перемещение папки	<code>Directory.Move(path, "куда");</code>	<code>di.MoveTo("куда");</code>
Получение родительской папки	<code>Directory.GetParent(path);</code> <code>DirectoryInfo directory = Directory.GetParent(path);</code>	Свойство di.Parent <code>DirectoryInfo directory = di.Parent;</code>
Проверка существования папки	<code>Directory.Exists(path);</code> <code>if (Directory.Exists(path)) { // работа с существующей папкой }</code>	Свойство di.Exists <code>if (di.Exists) { // работа с существующей папкой }</code>
Получение списка папок	<code>Directory.GetDirectories(path);</code> Возвращает массив названий папок <code>string[] directories = Directory.GetDirectories(path);</code>	<code>di.GetDirectories();</code> Возвращает массив объектов DirectoryInfo <code>DirectoryInfo[] directories = di.GetDirectories();</code>
Получение списка файлов	<code>Directory.GetFiles(path);</code> Возвращает массив названий файлов <code>string[] files = Directory.GetFiles(path);</code>	<code>di.GetFiles();</code> Возвращает массив объектов FileInfo <code>FileInfo[] files = di.GetFiles();</code>

У методов **GetDirectories** и **GetFiles** есть дополнительные параметры:

- **string searchPattern** – шаблон для поиска (например: *.txt – все названия, заканчивающиеся на .txt; LabWork* - все названия, начинающиеся на LabWork; в шаблоне * - любые символы)
- **SearchOption searchOption** – опция поиска:
 - **SearchOption.AllDirectories** – поиск по папке и вложенным папкам
 - **SearchOption.TopDirectoryOnly** – поиск только по папке без вложенных папок (по умолчанию)

Пример 2. Получение информации о папке с проверкой ее существования:

Добавить на форму метку с названием directoryLabel. На загрузку формы прописать следующий код:

```
string path = @"C:\Temp";
DirectoryInfo directory = new DirectoryInfo(path);
if (directory.Exists)
{
    directoryLabel.Text = $"Название каталога: {directory.Name} " +
        $"Полное название каталога: {directory.FullName} " +
        $"Время создания каталога: {directory.CreationTime}";
}
else
{
    directoryLabel.Text = $"Папка {path} не существует";
}
```

3 Работа с файлами. Классы File и FileInfo

Класс File – предоставляет набор статических методов для работы с файлом и получения информации о файле.

Общая форма вызова методов:

File.ИмяМетода("имя файла", параметры метода)

Первым параметром передается имя файла, например: "C:\\Temp\\1.txt" или @"C:\\Temp\\1.txt".

Класс FileInfo – предоставляет набор методов экземпляра для работы с файлом и свойств для получения информации о файле.

Для использования обязательно создать объект класса:

FileInfo имяОбъекта = new FileInfo("имя файла");

имяОбъекта.ИмяМетода(параметры метода) или **имяОбъекта.ИмяСвойства**

Имя файла указано при создании объекта, поэтому в параметрах метода не указывается

Примеры методов и свойств классов File и FileInfo и их вызовов

Действие	Применение File	Применение FileInfo
Предварительное действие для работы примеров	Создать строковую переменную fileName для хранения имени файла: <code>string fileName = @"C:\Temp\1.txt";</code>	Создать переменную file типа FileInfo: <code>string fileName = @"C:\Temp\1.txt"; FileInfo file = new FileInfo(fileName);</code>
Создание файла	<code>File.Create(fileName);</code>	<code>file.Create();</code>
Удаление файла	<code>File.Delete(fileName);</code>	<code>file.Delete();</code>
Копирование файла	<code>File.Copy(filename, "новое имя");</code>	<code>file.CopyTo("новое имя");</code>
Перемещение файла	<code>File.Move(fileName, "новое имя");</code>	<code>file.MoveTo("новое имя");</code>
Проверка существования файла	<code>File.Exists(fileName);</code> <code>if (File.Exists(fileName)) { // работа с существующим файлом }</code>	Свойство Exists <code>if (file.Exists) { // работа с существующим файлом }</code>

Свойства переменной типа FileInfo для получения информации о файле:

- **Name** – имя файла (без пути)
- **Extension** – расширение файла
- **FullName** – полное имя файла (путь+имя файла)
- **DirectoryName** – путь к файлу (без имени)
- **Length** – размер файла

Класс FileStream представляет возможности по считыванию из файла и записи в файл (двоичный или текстовый). Файловый поток **FileStream** можно создать, используя File.Open(...):

File.Open(имя файла, FileMode.ВариантОткрытия)

Варианты открытия файла:

- FileMode.Append – открытие на дозапись
- FileMode.Open – открытие существующего файла
- FileMode.Create – создание файла, существующий перезапишется
- FileMode.CreateNew – создание файла, если такой не существует
- FileMode.OpenOrCreate – открытие существующего или создание нового файла